

Public Key Algorithms

Dr. Mohammed Misbahuddin

PKI Module, DITISS Course

C-DAC, Electronics City, Bangalore

Overview



- Advanced Encryption Standard (AES)
- Public key algorithms
 - Diffie-Hellman Key Exchange,
 - RSA
- Digital Signature Algorithms
 - DSA

Block Ciphers - AES



- Advanced Encryption Standard
 - National Institute of Standards and Technology search for standard replacement for DES
 - Requirements:
 - 128 bit blocksize
 - 128, 192, 256 bit keys
 - Finalists announced in August 1999
-

Block Ciphers - AES



- Serpent
 - Rijndael
 - Twofish
 - Mars
 - RC6
 - Winner: Rijndael
-

Block Ciphers - AES



- Joan Daemen and Vincent Rijmen (Belgium)
 - Based on an algorithm called *Square*
 - Supports keys which are multiples of 32 bits and block sizes which are multiples of 64 bits
 - Number of rounds changes based on key size
 - Generally faster than the other candidates
-

Block Ciphers - AES



- Key Schedule for Rijndael:
 - Expand cipher key (varies depending on key length)
 - Round keys are taken from the expanded cipher key
 - Round keys are then rotated, passed through the S-box, and XOR'd with a round dependent constant (constants based on similar computation to S-box)

Block Ciphers - AES



- Variable rounds:
 - 9 if both block and key are 128 bits
 - 11 if either block or key is 192 bits and neither are longer than that
 - 13 if either block or key is 256 bits
 - etc

Block Ciphers - AES



- Using Rijndael:
 1. Perform Add Round Key Step (XOR a subkey with the block)
 2. Perform rounds:
 1. Byte Sub (each byte of the block is replaced by its substitute from an S-box)
 2. Shift Row
 1. Bytes are arranged in a rectangle and shifted,

ex.	from	to
	1 5 9 13	1 5 9 13
	2 6 10 14	6 10 14 2
	3 7 11 15	11 15 3 7
	4 8 12 16	16 4 8 12

Block Ciphers - AES



- The S-box is:

99	124	119	123	242	107	111	197
48	1	103	43	254	215	171	118
202	130	201	125	250	89	71	240
173	212	162	175	156	164	114	192
183	253	147	38	54	63	247	204
52	165	229	241	113	216	49	21
4	199	35	195	24	150	5	154
7	18	128	226	235	39	178	117
9	131	44	26	27	110	90	160
82	59	214	179	41	227	47	132
83	209	0	237	32	252	177	91
106	203	190	57	74	76	88	207
208	239	170	251	67	77	51	133
69	249	2	127	80	60	159	168
81	163	64	143	146	157	56	245
188	182	218	33	16	255	243	210
205	12	19	236	95	151	68	23
196	167	126	61	100	93	25	115
96	129	79	220	34	42	144	136
70	238	184	20	222	94	11	219
224	50	58	10	73	6	36	92
194	211	172	98	145	149	228	121
231	200	55	109	141	213	78	169
108	86	244	234	101	122	174	8
186	120	37	46	28	166	180	198
232	221	116	31	75	189	139	138
112	62	181	102	72	3	246	14
97	53	87	185	134	193	29	158
225	248	152	17	105	217	142	148
155	30	135	233	206	85	40	223
140	161	137	13	191	230	66	104
65	153	45	15	176	84	187	22

Block Ciphers - AES



3. Mix Column

- Matrix multiplication performed where each column is multiplied by:

2 3 1 1

1 2 3 1

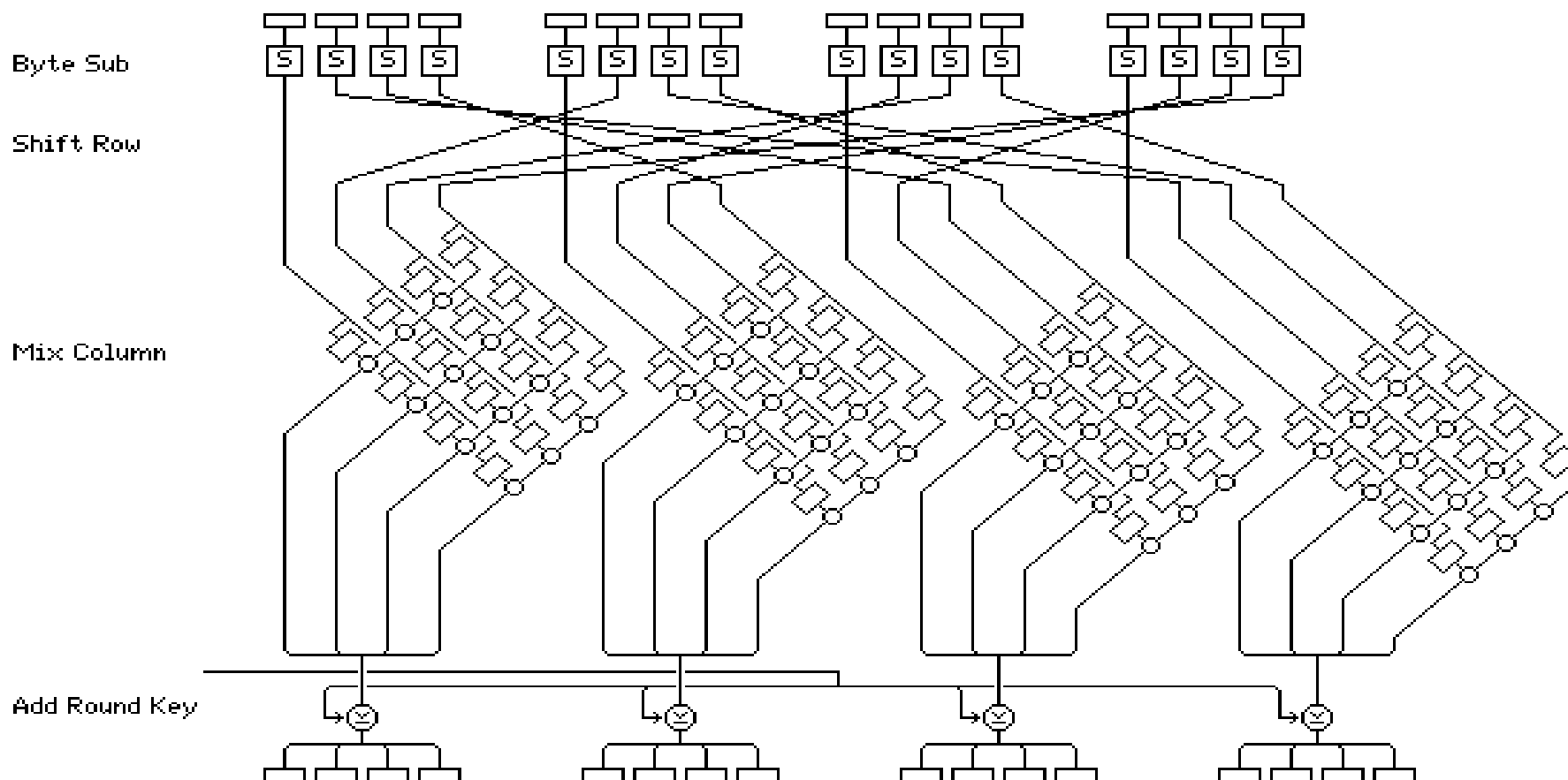
1 1 2 3

3 1 1 2

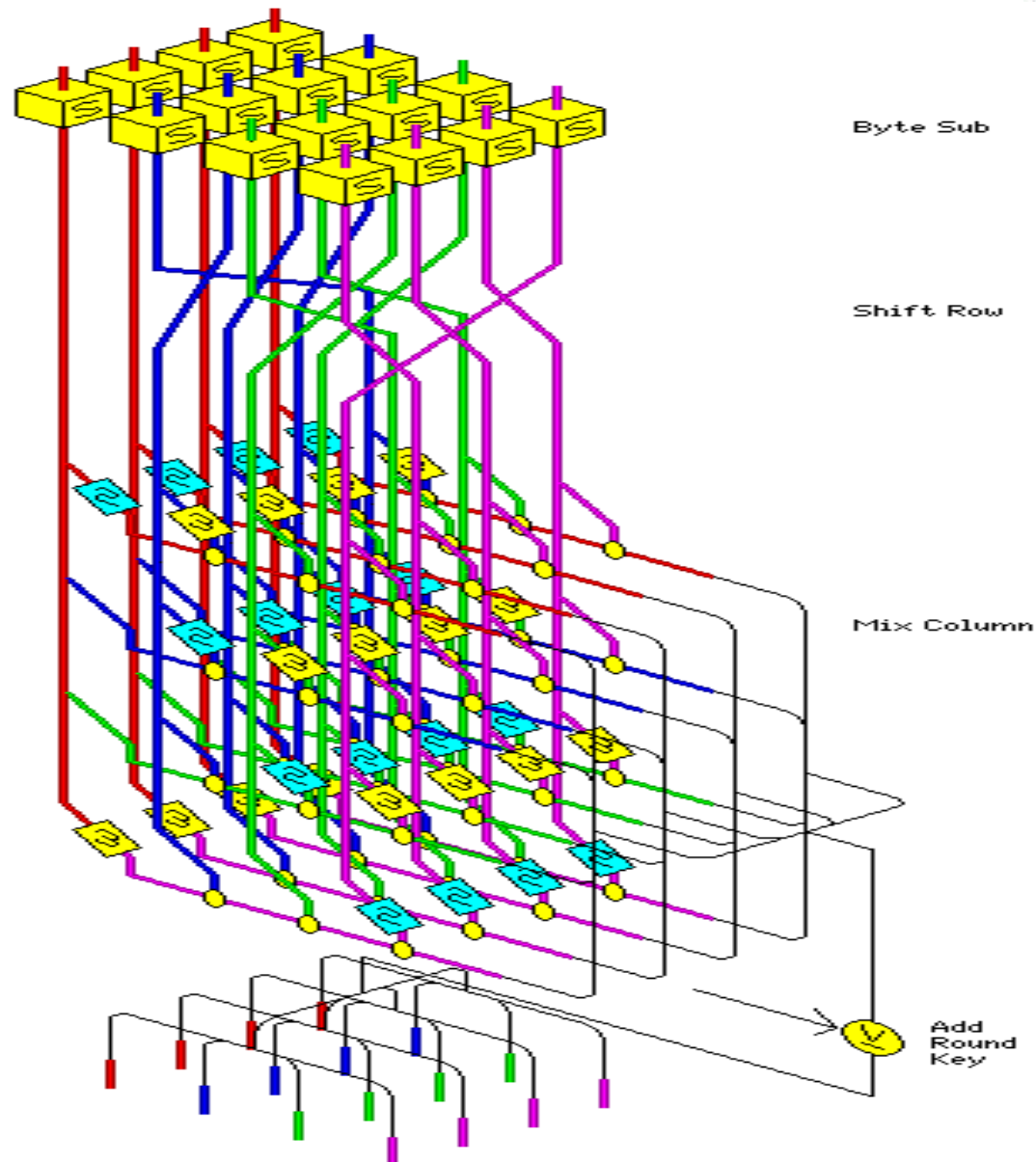
4. Add round key (XOR subkey for current round)

- » An extra final round is added where the mix column step is omitted

Block Ciphers - AES



Block Ciphers - AES



Diffie-Hellman Key Exchange

Modular Arithmetic



- If current time is 9 o'clock, after 4 hours the clock will show...
 - It is not at 13 o'clock, but it is 1 o'clock.
 - When we reach 12, we start over; 12 is called the modulus.
- Example
 - $10 \bmod 6 = 4$ $5 \bmod 9 = 5$
 - $32 \bmod 8 = 0$

Diffie-Hellman Key Exchange



- Public key algorithm for key exchange
- Allows two users to exchange a secret key over an insecure medium without any prior secrets.
 - Functionality limited to key exchange only
- The scheme was first published by Whitfield Diffie and Martin Hellman in 1976.

Diffie-Hellman Key Exchange



Jai

Insecure Medium

Veeru

Select numbers

n, g and x

Calculate $A_k = g^x \bmod n$

Sending n, g and A_k

(n, g, A_k)

Select number y

Calculate $B_k = g^y \bmod n$

Sending B_k

B_k

Calculating Shared Key K_s

$K_s = B_k^x$

Calculating Shared Key K_s

$K_s = A_k^y$

Diffie-Hellman Key Exchange



$$n=97 \quad g=5$$

Jai

$$X=36$$

$$\begin{aligned} A_k &= 5^{36} \bmod 97 \\ &= 50 \bmod 97 = 50 \end{aligned}$$

44

$$44^{36} \bmod 97 = 75 \bmod 97$$

Secret Key=75

Insecure Channel

97, 5, 50

44

Veeru

$$Y=58$$

$$\begin{aligned} B_k &= 5^{58} \bmod 97 \\ &= 44 \bmod 97 = 44 \end{aligned}$$

$$50^{58} \bmod 97 = 75 \bmod 97$$

Secret Key=75

MITM on DHKE



Jai

Gabbar

Veeru

$N = 97$ and $g = 5$

Private Key $X_a = 36$

Public Key $Y_a = 5^{36} \bmod 97$
 $= 50 \bmod 97$

97, 5, 50

$79^{36} \bmod 97 =$
 $22 \bmod 97$
Shared Key = 22

Private Key $X_v = 25$

Public Key $Y_v = 5^{25} \bmod 97$
 $= 13 \bmod 97$

97, 5, 13

Private Key $X_J = 30$

Public Key $Y_J = 5^{30} \bmod 97$
 $= 79 \bmod 97$

79

Key with Jai = $50^{30} \bmod 97 =$
 $22 \bmod 97 = 22$

Key with Veeru = $44^{25} \bmod 97$
 $= 53 \bmod 97 = 53$

Private Key $X_b = 58$

97, 5, 13
Public Key $Y_b = 5^{58} \bmod 97$
 $= 44 \bmod 97$

44

$13^{58} \bmod 97 = 53 \bmod 97$

Shared Key = 53

Limitation of Diffie- Hellman Algorithm



- ☹ Cannot be used for Encryption/Decryption and Digital Signature
- ☹ Does not provide authentication to the communication parties.
 - ☹ Vulnerable to man-in the middle attack

RSA

RSA Algorithm



➡ RSA

- ➡ Developed by Ronald Rivest, Adi Shamir and Leonard Adleman

➡ Functionality

- ➡ Key Generation
- ➡ Encryption
- ➡ Decryption
- ➡ Digital signatures

➡ Based on number theory

RSA Algorithm



- Key Generation Steps
 - Choose two large prime numbers p and q
 - Compute n and z such that $n=p*q$ and $z=(p-1)*(q-1)$
 - Choose a number d relatively prime to z
 - Compute e such that $(e*d) = 1 \bmod z$

Public Key (n, e)

Private Key (n, d)

RSA Algorithm



- Encryption

- ▢ Message 'm', cipher text 'c'

- ▢ $c = m^e \bmod n$

- ▢ where (n,e) is receivers public key

- Decryption

- ▢ $m = c^d \bmod n$

- where (n,d) receivers private key

RSA – Key Generation Example



- Select primes $p=11$, $q=3$.
- $n = p * q = 11 * 3 = 33$ $n=p*q$
- $z = (p-1)(q-1) = 10 * 2 = 20$ $z=(p-1)*(q-1)$
- Choose $d=7$
- Compute 'e' such that $(7 * e) = 1 \text{ mod } 20$
 $e=3$ $(e*d) = 1 \text{ mod } z$
- Public key = $(n, e) = (33, 3)$
- Private key = $(n, d) = (33, 7)$.

RSA – Encryption and Decryption



- Encrypt the message $m = 7$

$$c = m^e \bmod n = 7^3 \bmod 33 = 343 \bmod 33$$

cipher text $c = 13$

- Decryption

$$m' = c^d \bmod n = 13^7 \bmod 33 = 7$$

Example



- An example of the RSA algorithm.

Plaintext (P)		Ciphertext (C)			After decryption	
<u>Symbolic</u>	<u>Numeric</u>	<u>P³</u>	<u>P³ (mod 33)</u>	<u>C⁷</u>	<u>C⁷ (mod 33)</u>	<u>Symbolic</u>
S	19	6859	28	13492928512	19	S
U	21	9261	21	1801088541	21	U
Z	26	17576	20	1280000000	26	Z
A	01	1	1	1	01	A
N	14	2744	5	78125	14	N
N	14	2744	5	78125	14	N
E	05	125	26	8031810176	05	E
Sender's computation				Receiver's computation		

ECC Vs RSA



ECC Key Size (Bits)	RSA Key size (Bits)	Key Size Ratio
163	1024	1:6
256	3072	1:12
384	7680	1:20
512	15360	1:30

Reference : NIST Recommendation for Key Management – Part 1: General
(Revised)

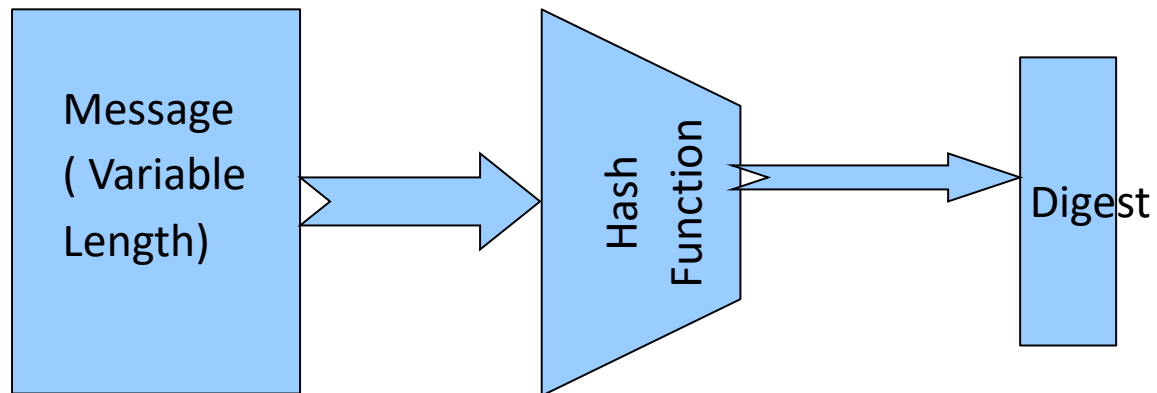
Hash Functions

Hash Function



- A hash function is an algorithm
 - which creates a digital representation or "fingerprint" in the form of a "hash value" or "hash result" of a standard length
 - Hash value is usually much smaller than the message but nevertheless substantially unique to it.
 - Any change to the message invariably produces a different hash result when the same hash function is used
 - It is a one way function – Easy to calculate hash value from message but difficult to generate message from hash value

Hash Function



Message Digest Algorithm (MD5)



- Message Digest Hashing Algorithm
- Developed by Ronald Rivest
- Processing input text in 512 bit block
- Output size 128 bit
- Specified as Internet standard RFC1321

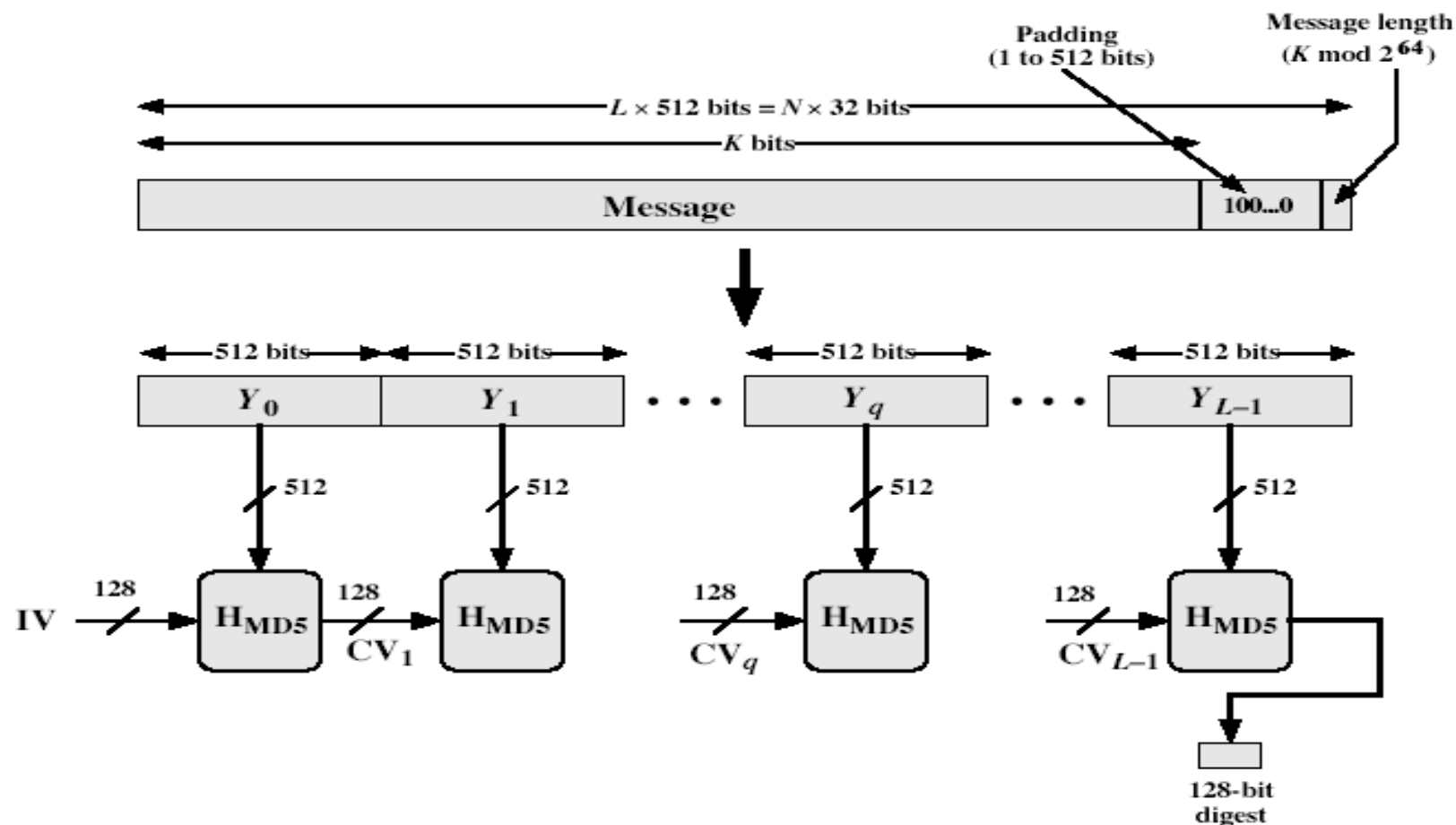
MD5 Algorithm



Message Digest Algorithm - Steps

1. Pad message so its length is $448 \bmod 512$
2. Append a 64-bit length value to message
3. Initialise 4-word (128-bit) MD buffer (A,B,C,D)
4. process message in 16-word (512-bit) blocks:
 - Compression function consists of 4 rounds of 16 steps on message block & buffer
 - add output to buffer input to form new buffer value
5. output hash value is the final buffer value

MD5 Algorithm



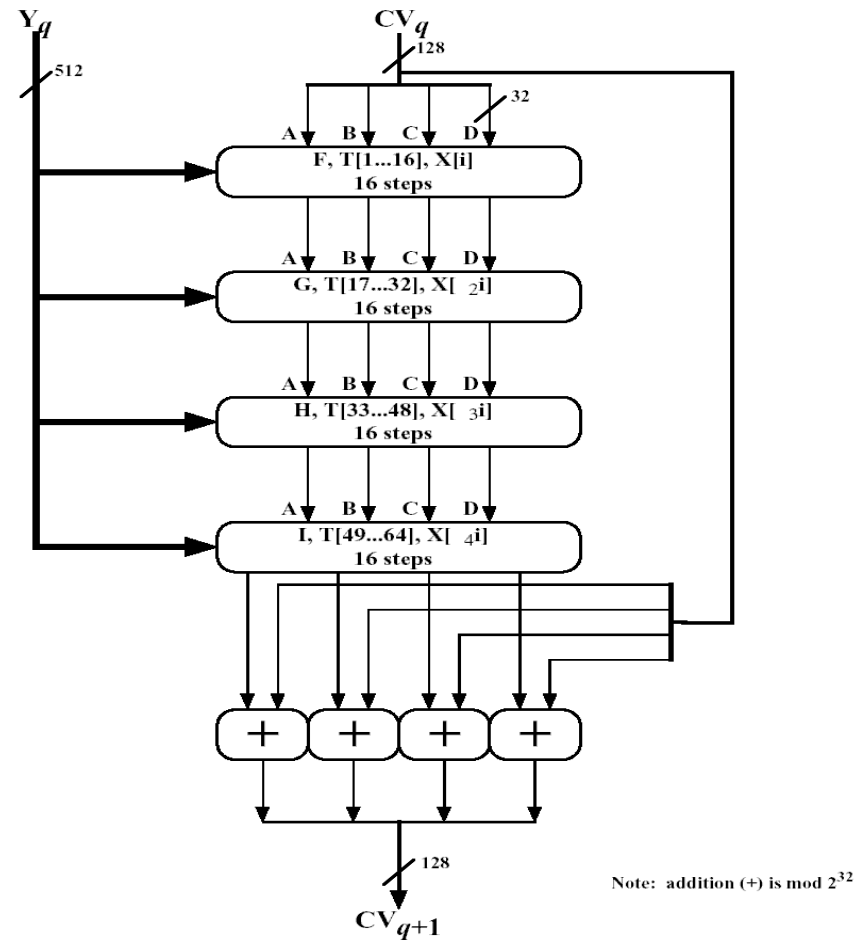
Message Digest Generation using MD5

MD5 compression

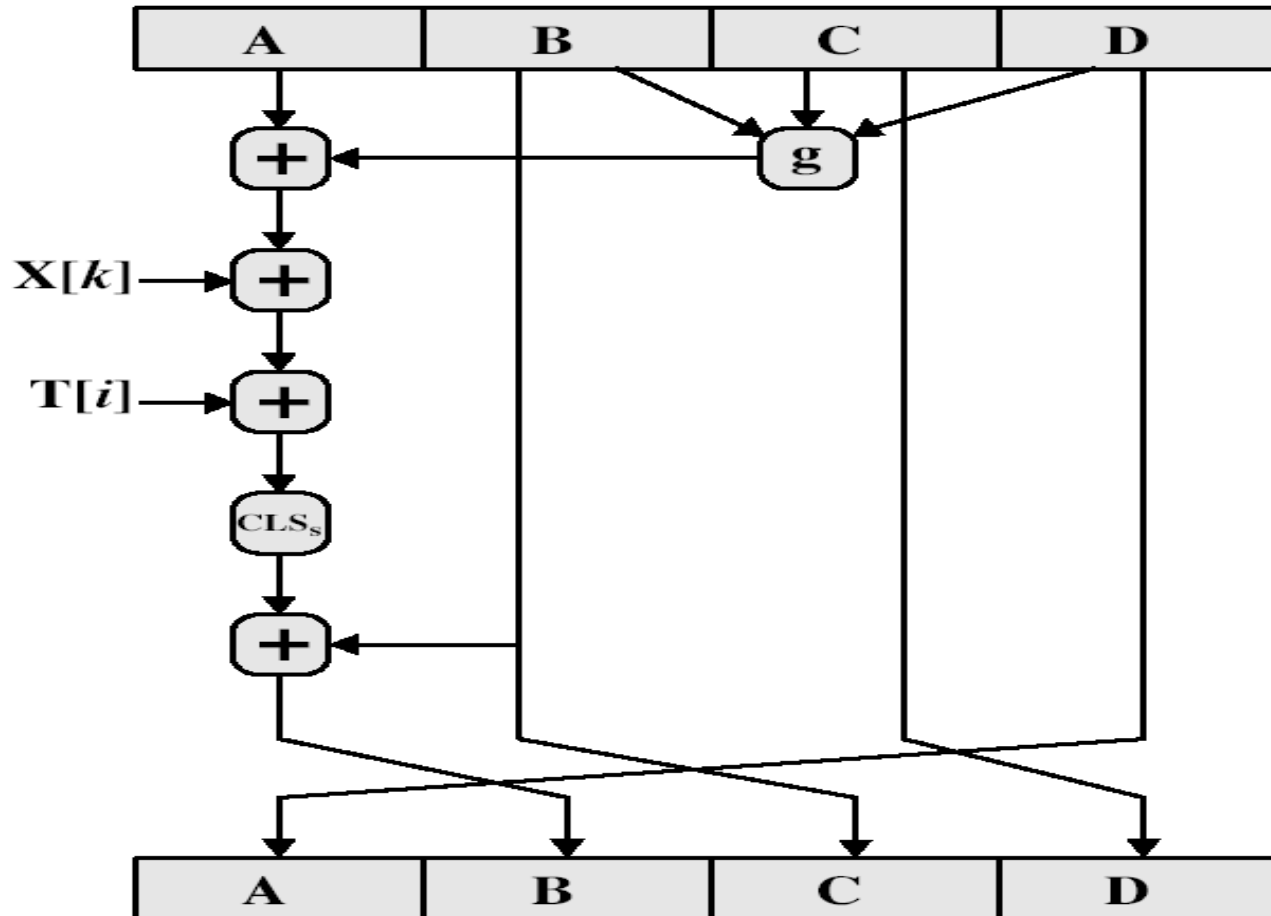


- Each round has 16 steps of the form:
 - $a = b + ((a + g(b, c, d) + X[k] + T[i]) \ll s)$
- a, b, c, d refer to the 4 words of the buffer, but used in varying permutations
 - After 16 steps each word is updated 4 times
- where $g(b, c, d)$ is a different nonlinear function in each round (F, G, H, I)
- $X[k] = M[q * 16 + k]$ = k th 32 bit word in q th 512 bit block of message
- $T[i]$ is a constant value derived from sine function

MD5 Single block Processing



Elementary MD5 operation



Secure Hash Algorithm (SHA)



- SHA was designed by NIST & NSA in 1993, revised 1995 as SHA-1
- US standard for use with DSA signature scheme
 - standard is FIPS 180-1 1995, also Internet RFC3174
 - the algorithm is SHA, the standard is SHS
- produces 160-bit hash values
- now the generally preferred hash algorithm

SHA - steps



1. Pad message so its length is $448 \bmod 512$
2. Append a 64-bit length value to message
3. Initialize 5-word (160-bit) buffer (A,B,C,D,E) to (67452301,efcdab89,98badcfe,10325476,c3d2e1f0)
4. Process message in 16-word (512-bit) chunks:
 - expand 16 words into 80 words by mixing & shifting
 - use 4 rounds of 20 bit operations on message block & buffer
 - add output to input to form new buffer value
5. Output hash value is the final buffer value

SHA Vs MD5



- SHA
 - Brute force attack is harder (160 Vs 128 bits for MD5)
 - Not vulnerable to any known cryptanalytic attacks (compared to MD4/5)
 - A little slower than MD5 (80 Vs 64 steps)
- Both work well on a 32-bit architecture
- Both designed as simple and compact for implementation

Digital Signature Standards (DSS)



- DSS is a standard for Digital Signature
- The DSS defines a public key cryptographic system for generating and verifying digital signatures
- Digital signature is represented as a string of bits
- Digital signature(DS) is computed using set of rules and set of parameters such that the identity of the signatory and integrity of the data can be verified
- DS can be generated for stored data and programs . So integrity of the data can be verified in later time

Digital Signatures (DS)



- DS contains two process
 - Signature Creation
 - Signature Verification
- Signature Creation
 - Performed by sender
 - Create the hash result (digest) of the message
 - Transform hash result into digital signature using senders private key

Digital Signatures



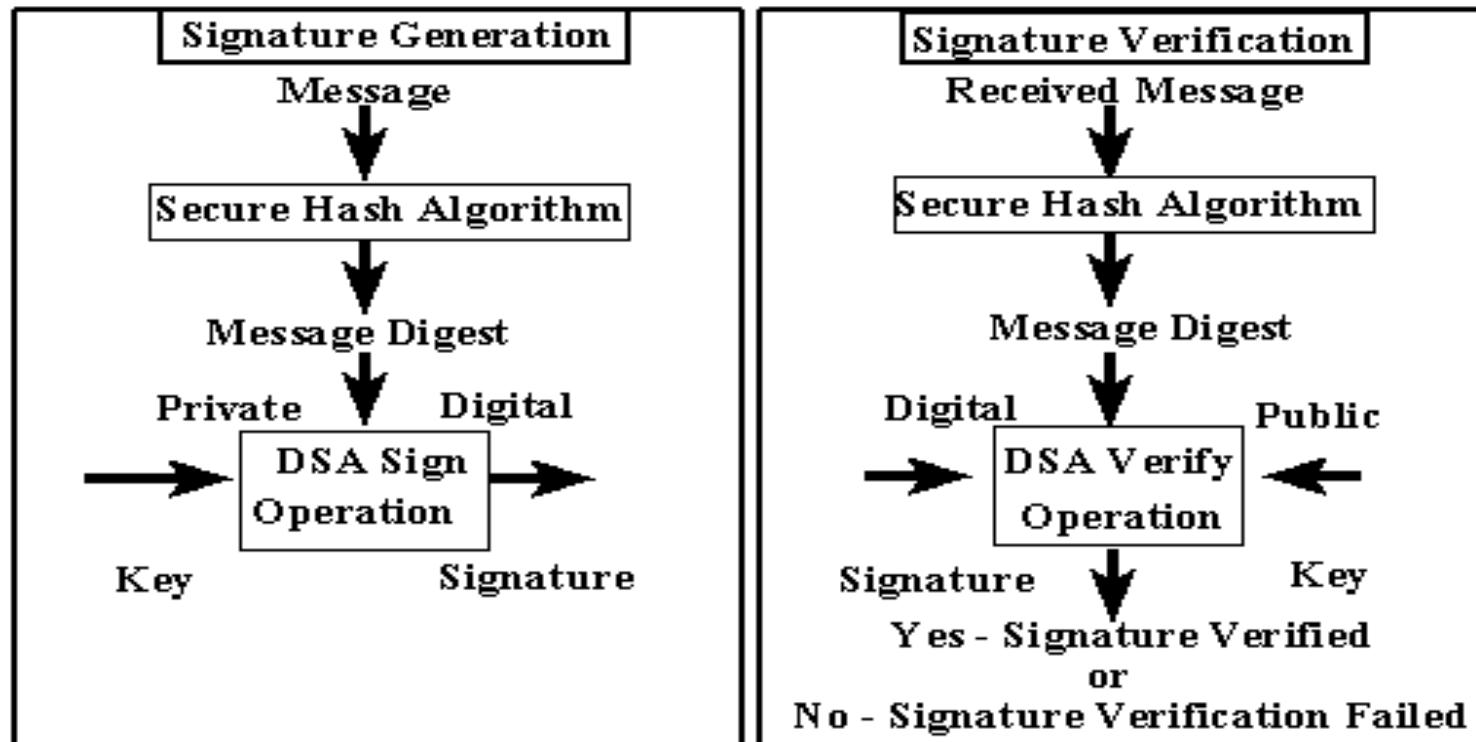
- Signature Verification
 - Performed by receiver
 - Verify the signature using senders public key
 - Calculate the hash of received message using same hash function used by the sender

Digital Signature Algorithm



- Digital Signature Algorithm is defined in Digital Signature Standards (DSS)
- DSA provides the capability to generate and verify signatures
- A hash function is used in the signature generation process to obtain a condensed version of data, called a message digest .
- The message digest is then input to the DSA to generate the digital signature

Digital Signature Algorithm



References



- Cryptography and Network security – principles and practice : William Stallings
- Applied Cryptography, Second Edition: Protocols, Algorithms, and Source Code in C : Bruce Schneier
- www.certicom.com/index.php/ecc-tutorial
- http://campustechnology.com/articles/39190_2
- <http://csrc.nist.gov/>
- <http://www-fs.informatik.uni-tuebingen.de/~reinhard/krypto/English/english.html>